1. Tại sao không dùng Monolith mà lại chia Microservices?

1. Lý do cốt lõi: Đề tài yêu cầu mật mã cho giao tiếp giữa các dịch vụ

Đây là lý do **mạnh nhất** và **khó phản bác nhất**:

Đề tài môn Cryptography yêu cầu triển khai và đánh giá:

- **mTLS** (mutual TLS) cho giao tiếp service-to-service
- **HMAC signing** cho request giữa các dịch vụ
- **API Gateway** làm điểm kiểm soát trung tâm

Nếu dùng monolith: tất cả module nằm trong cùng một process → khi Cart gọi Payment, đó chỉ là một **function call trong bộ nhớ → không có đường truyền mạng nào** để áp dụng mTLS, HMAC, hay API Gateway cả.

```
MONOLITH (1 process):
┌──────────────────────────────────┐
│  Cart.checkout()                 │
│    → Payment.charge()  ← gọi     │  ← Chỉ là function call
│    → Order.create()      hàm     │  ← Không qua mạng
│    → Email.send()        nội bộ  │  ← Không có gì để mã hóa
└──────────────────────────────────┘

MICROSERVICES (nhiều process):
┌─────────┐   ──mTLS──▶   ┌─────────┐
│  Cart   │               │ Payment │   ← Qua mạng, có thể
└─────────┘   ◀──HMAC──   └─────────┘   ← bị nghe lén, giả mạo
     │                                   ← → CẦN mTLS, HMAC
     │──mTLS──▶   ┌───────┐
     │            │ Order │
     └──────────▶ └───────┘
```

> **Câu trả lời ngắn gọn:** "Nếu dùng monolith, sẽ không có giao tiếp mạng giữa các module → không thể triển khai và đánh giá mTLS, HMAC, API Gateway"

2. Lý do thứ hai: Cách ly vùng bảo mật (Security boundary isolation)

Trong thương mại điện tử thực tế, **Payment Service** xử lý dữ liệu thẻ — cực kỳ nhạy cảm. PCI-DSS (Payment Card Industry Data Security Standard) yêu cầu:

- Hệ thống xử lý dữ liệu thẻ phải được **cách ly** khỏi phần còn lại
- Chỉ những thành phần **cần thiết** mới được truy cập vào vùng này

Kiến trúc	Cách ly Payment	Tuân thủ PCI-DSS
Monolith	Không thể — tất cả module cùng process, cùng bộ nhớ. Nếu Catalog bị hack → attacker ở trong cùng process với Payment → truy cập được dữ liệu thẻ	**Toàn bộ ứng dụng** nằm trong phạm vi PCI-DSS → phải audit toàn bộ

Kiến trúc	Cách ly Payment	Tuân thủ PCI-DSS
Microservices	Payment Service chạy riêng, mạng riêng, DB riêng. Catalog bị hack → attacker vẫn phải vượt qua mTLS + firewall mới tới Payment	**Chỉ Payment Service** nằm trong phạm vi PCI-DSS → audit ít hơn, an toàn hơn

> **Câu trả lời:** "Microservices cho phép cách ly Payment Service vào vùng bảo mật riêng, thu hẹp phạm vi PCI-DSS. Với monolith, một lỗ hổng ở bất kỳ module nào cũng có thể ảnh hưởng đến dữ liệu thanh toán."

---

3. Lý do thứ ba: Nguyên tắc Least Privilege

Mỗi microservice chỉ có **quyền tối thiểu**:

```
Catalog Service:  chỉ READ database sản phẩm, không có quyền gì với DB thanh toán
Cart Service:     chỉ READ/WRITE session user, không truy cập KMS
Payment Service:  có quyền gọi KMS, gọi PSP, nhưng không truy cập catalog
```

Với monolith, tất cả module chạy dưới **cùng một identity / cùng quyền** → nếu một module bị khai thác, attacker có toàn bộ quyền của ứng dụng.

---

4. Lý do thứ tư: Thí nghiệm bảo mật đòi hỏi ranh giới mạng

5 thí nghiệm của đề tài hầu hết cần **nhiều thực thể riêng biệt giao tiếp qua mạng**:

Thí nghiệm	Cần microservices vì...
Token replay	Cần API Gateway riêng để validate/reject token
Payment fraud	Cần Payment Service riêng gọi PSP qua mạng → có thể intercept, test 3DS
API abuse	Cần rate limiting **tại API Gateway** — monolith không có gateway
Key rotation	Cần nhiều service dùng chung KMS → test xem service nào bị ảnh hưởng khi xoay khóa
Supply chain	Cần **nhiều container image** riêng biệt để test artifact signing

---

5. Nhưng cũng phải thành thật: Trade-off

Nếu giáo viên hỏi tiếp *"Microservices có nhược điểm gì không?"*:

Nhược điểm của Microservices	Cách đề tài xử lý
Phức tạp hơn monolith rất nhiều (deploy, debug, monitor)	Dùng Kubernetes + Helm để quản lý tự động
Latency cao hơn vì giao tiếp qua mạng thay vì function call	Đề tài đo latency này như một phần thí nghiệm hiệu năng (mục 8.3) — đây chính là trade-off cần đánh giá
Quản lý dữ liệu phân tán, khó đảm bảo consistency	Dùng saga pattern / idempotency key cho payment flow
Tốn tài nguyên hơn (nhiều container, mỗi cái có overhead riêng)	Chấp nhận được trong phạm vi lab (k3s đủ nhẹ)

> **Câu trả lời mẫu:** "Em nhận thức được microservices phức tạp hơn monolith về vận hành. Tuy nhiên trong ngữ cảnh đề tài Cryptography, sự phức tạp này chính là **đối tượng nghiên cứu** — em cần đánh giá trade-off giữa bảo mật tăng thêm và chi phí hiệu năng/vận hành. Nếu dùng monolith, em sẽ không thể thực hiện được các thí nghiệm này."

Tóm tắt câu trả lời hoàn chỉnh cho giáo viên

> *"Lý do chính là đề tài yêu cầu triển khai và đánh giá mTLS, HMAC, API Gateway cho giao tiếp giữa các thành phần — những cơ chế mật mã này chỉ có ý nghĩa khi các thành phần giao tiếp qua mạng, tức microservices. Ngoài ra, microservices cho phép cách ly vùng bảo mật (đặc biệt Payment Service theo yêu cầu PCI-DSS) và áp dụng nguyên tắc least privilege cho từng service. Em cũng nhận thức rằng microservices tạo thêm phức tạp và latency, nhưng đây chính là trade-off mà đề tài yêu cầu em đo lường và đánh giá."*

2. Tech-stack

Bảng tổng hợp

Thành phần	Chọn	Lý do chọn cái này
Orchestration	**k3s**	Nhẹ hơn minikube, chạy multi-node được, gần production hơn
API Gateway	**Envoy**	Tài liệu nhắc Envoy trong mục 8.1, phổ biến trong hệ sinh thái K8s hơn, là core của Istio service mesh (hỗ trợ mTLS tự động)
Identity Provider	**Keycloak**	Mã nguồn mở, self-hosted (chạy trong lab được), tài liệu nhắc Keycloak 3 lần trong đề bài (mục 8.1, 11, 16)
Payment Sandbox	**Stripe test mode**	Docs tốt nhất ngành, SDK đa ngôn ngữ, 3DS test cards có sẵn.
Key Management	**HashiCorp Vault**	Dùng cho lab (Tài liệu ghi rõ mục 6.1: "local KMS emulator - HashiCorp Vault"). Vault vừa làm KMS emulator, vừa quản lý secrets, vừa Transit engine thay HSM
Database	**PostgreSQL**	Tài liệu chỉ định rõ
Event Streaming	**Kafka**	Tài liệu chỉ định rõ trong mục 5.8
Monitoring (metrics)	**Prometheus + Grafana**	Tài liệu.
Monitoring (logs)	**ELK** (Elasticsearch + Logstash + Kibana)	Tài liệu chỉ định rõ trong mục 6.2 và 15
Load Testing	**JMeter**	Có UI, dễ demo kết quả
Security Testing	**OWASP ZAP**	Miễn phí hoàn toàn (Burp bản đầy đủ trả phí)
Dependency Scanning	Dependabot	Dependabot tích hợp sẵn GitHub (miễn phí).
Ngôn ngữ services	**Tùy (Python/FastAPI)**	Tài liệu để mở
Frontend	**cái này thì vibe code**	Chỉ cần đủ demo là được

3. Timeline

Tuần 1: Nền tảng

```
┌──────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Architecture doc + Lab infra chạy được        │
├──────────────────────────┬───────────────────────────────┤
│        NGƯỜI A           │          NGƯỜI B              │
├──────────────────────────┼───────────────────────────────┤
│ □ Vẽ Architecture Diagram │ □ Viết Threat Model (STRIDE)  │
│   (draw.io / Excalidraw)  │   cho từng component          │
│                           │                               │
│ □ Setup Docker Compose:   │ □ Viết Security Design Doc:   │
│   - k3s cluster           │   - Key hierarchy diagram     │
│   - Keycloak              │   - Data flow (checkout)      │
│   - PostgreSQL            │   - Token flow (OAuth2)       │
│   - Vault (dev mode)      │   - Payment flow (Stripe)     │
│                           │                               │
│ □ Keycloak: tạo realm,    │ □ Tạo GitHub repo với câú     │
│   client, test user,      │   trúc thư mục chuẩn          │
│   test lấy token          │   (theo mục 16 đề`bài)        │
├──────────────────────────┴───────────────────────────────┤
│  ✅ CHECKPOINT: docker-compose up → Keycloak + DB + Vault  │
│  chạy được. Architecture doc hoàn chỉnh.                  │
│  → Đây chính là deliverable MID-TERM                      │
└──────────────────────────────────────────────────────────┘
```

Tuần 2 và 3: Code Core Services

```
┌──────────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: 4 services chính + API Gateway + Auth hoạt động │
├──────────────────────────────┬──────────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B                 │
├──────────────────────────────┼──────────────────────────────────┤
│                      TUẦN 2                                      │
│                                                                  │
│ □ Setup Envoy/Istio          │ □ Viết Catalog Service           │
│   trên k3s                   │   - GET /products                │
│   - TLS termination          │   - Kết nối PostgreSQL           │
│   - JWT validation plugin    │   - JWT validation middleware    │
│   - Rate limiting            │                                  │
│                              │ □ Viết Cart Service              │
│ □ Cấu hình mTLS tự động      │   - POST/GET /cart               │
│   giữa services (Istio)      │   - Authenticated endpoints      │
│                              │                                  │
├──────────────────────────────┼──────────────────────────────────┤
│                      TUẦN 3                                      │
│                                                                  │
│ □ Vault Transit Engine:      │ □ Viết Order Service             │
│   - Encryption-as-service    │   - POST /checkout               │
│   - Tạo policies cho         │   - Orchestrate Cart→Payment     │
│     từng service             │                                  │
│                              │ □ Viết Payment Service           │
│ □ Vault PKI Engine:          │   - Tích hợp Stripe sandbox      │
│   - CA nội bộ cho mTLS       │   - Tokenization flow            │
│   - Auto cert rotation       │   - 3DS test flow                │
│                              │   - Idempotency key              │
├──────────────────────────────┴──────────────────────────────────┤
│  ✅ CHECKPOINT: Chạy được flow hoàn chỉnh:                       │
│  Login (Keycloak) → Browse catalog → Add to cart →               │
│  Checkout → Pay (Stripe sandbox) — tất cả qua Envoy,             │
│  qua mTLS, authenticated bằng JWT                                │
└──────────────────────────────────────────────────────────────────┘
```

Tuần 4: Encryption + 3 services phụ

```
┌──────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Mã hóa dữ liệu at-rest + services phụ            │
├──────────────────────────────┬───────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B              │
├──────────────────────────────┼───────────────────────────────┤
│ □ PostgreSQL TDE setup       │ □ Viết Inventory Service      │
│                              │   (basic stock management)    │
│ □ Field-level encryption     │                               │
│   cho PII (email, address)   │ □ Viết Shipping Service       │
│   qua Vault Transit          │   (stub — tạo shipping label) │
│                              │                               │
│ □ Backup encryption          │ □ Viết Notification Service   │
│   (keys managed via Vault)   │   (stub — log email/SMS)      │
│                              │                               │
│ □ HMAC request signing       │ □ Implement RBAC              │
│   giữa services              │   (OAuth2 scopes cho mỗi      │
│                              │    service)                   │
├──────────────────────────────┴───────────────────────────────┤
│  ✅ CHECKPOINT: Dữ liệu nhạy cảm được mã hóa at-rest.        │
│  7 services chạy. HMAC signing hoạt động.                    │
└──────────────────────────────────────────────────────────────┘
```

Tuần 5: Monitoring + Anti-Fraud

```
┌──────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Observability stack + Fraud detection hoạt động  │
├──────────────────────────────┬───────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B              │
├──────────────────────────────┼───────────────────────────────┤
│ □ Thêm Kafka vào cluster     │ □ ML Fraud Scoring stub       │
│   - Topic: checkout-events   │   - scikit-learn model đơn    │
│   - Topic: fraud-alerts      │     giản hoặc rule-based      │
│   - Topic: audit-logs        │   - Consume từ Kafka          │
│                              │    - Output: risk score       │
│ □ Setup ELK Stack            │                               │
│   - Elasticsearch            │ □ Signed audit logs           │
│   - Logstash (collect        │   - Mỗi event được ký số      │
│     từ tất cả services)      │   - Append-only vào Kafka     │
│   - Kibana dashboards        │                               │
│                              │                               │
│ □ Setup Prometheus +         │ □ Chạy baseline functional    │
│   Grafana                    │   test: end-to-end flow       │
│   - Metrics từ services      │   hoạt động đúng              │
│   - Dashboard checkout       │                               │
│     latency                  │                               │
├──────────────────────────────┴───────────────────────────────┤
│  ✅ CHECKPOINT: Toàn bộ hệ thống production-ready.           │
│  Grafana dashboard đẹp. Kibana có log. Fraud scoring chạy.  │
└──────────────────────────────────────────────────────────────┘
```

Tuần 6:

```
┌──────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Chạy 5 thí nghiệm bảo mật, ghi lại số liệu        │
├──────────────────────────────┬───────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B              │
├──────────────────────────────┼───────────────────────────────┤
│ □ Experiment 4:              │ □ Experiment 1:               │
│   Key Compromise &           │   Token Replay & Binding      │
│   Rotation Drill             │   - Lấy token hợp lệ          │
│   - Xoay master key          │   - Replay từ device khác     │
│   - Verify rollback          │   - Ghi: bị chặn? log?        │
│   - Đo downtime              │                               │
│                              │ □ Experiment 2:               │
│ □ Experiment 5:              │   Payment Fraud Simulation    │
│   Supply Chain Tamper        │   - Duplicate payment request │
│   - Inject modified image    │   - Test idempotency          │
│   - Test artifact verify     │   - Test fraud scoring block  │
│   - CI/CD gating             │                               │
│                              │ □ Experiment 3:               │
│                              │   API Abuse & Rate Limiting   │
│                              │   - Credential stuffing       │
│                              │   - Đo: bao nhiêu req bị      │
│                              │     chặn, response time       │
├──────────────────────────────┴───────────────────────────────┤
│  ⚠️ MỖI EXPERIMENT PHẢI CÓ:                                  │
│  1. Mục đích & giả thuyết                                    │
│  2. Setup (environment, tools)                               │
│  3. Procedure (từng bước)                                    │
│  4. Kết quả (screenshots, số liệu, bảng)                     │
│  5. Phân tích & kết luận                                     │
│  → Format như bài lab khoa học = điểm cao                    │
└──────────────────────────────────────────────────────────────┘
```

TUẦN 7 — Performance + Operational Drills

```
┌──────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Benchmark hiệu năng + drills vận hành             │
├──────────────────────────────┬───────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B              │
├──────────────────────────────┼───────────────────────────────┤
│ □ Operational drills:        │ □ JMeter load tests:          │
│   - Key rotation procedure   │   - Checkout latency          │
│   - Incident response        │     median/p95/p99            │
│     runbook                  │   - Throughput under load     │
│   - MTTR measurement         │                               │
│                              │ □ Ablation studies:           │
│ □ Cost analysis:             │   - Có mTLS vs không: latency?│
│   - KMS calls/10k req        │   - Vault encrypt vs local:   │
│   - HSM rental estimate      │     overhead?                 │
│   - So sánh software key     │   - Rate limiting on vs off:  │
│     vs HSM                   │     throughput?               │
│                              │                               │
│ □ CI/CD pipeline:            │ □ Tổng hợp tất cả kết quả     │
│   - GitHub Actions           │   thí nghiệm vào CSV/bảng     │
│   - Code signing             │                               │
│   - Dependabot + Snyk        │                               │
│   - SBOM generation          │                               │
├──────────────────────────────┴───────────────────────────────┤
│  ✅ CHECKPOINT: Có đầy đủ số´liệu cho report.                 │
│  Runbooks viết xong. CI/CD pipeline hoạt động.               │
└──────────────────────────────────────────────────────────────┘
```

Tuần 8:

```
┌──────────────────────────────────────────────────────────────────┐
│  MỤC TIÊU: Nộp bài hoàn chỉnh                                    │
├──────────────────────────────┬───────────────────────────────────┤
│        NGƯỜI A               │          NGƯỜI B                  │
├──────────────────────────────┼───────────────────────────────────┤
│ □ Đóng gói repo:             │ □ Viết Final Report:              │
│   - Docker Compose chạy      │   - Design & architecture         │
│     1 lệnh                   │   - Experiments & results         │
│   - Helm chart cho k3s       │   - Mitigations & runbooks        │
│   - README hướng dẫn         │   - Cost analysis                 │
│     reproduce                │   - Kết luận & recommendations    │
│                              │                                   │
│ □ Quay Demo Video:           │ □ Làm Slides:                     │
│   - Show checkout flow       │   - Architecture overview         │
│   - Show 2 experiments:      │   - Key experiments               │
│     token replay blocked     │   - Results & analysis            │
│     + fraud detection        │   - Demo highlights               │
│                              │                                   │
│ □ Chạy OWASP ZAP scan        │ □ Review toàn bộ code +           │
│   → thêm kết quả vào         │   docs lần cuối                   │
│   report                     │                                   │
├──────────────────────────────┴───────────────────────────────────┤
│  ✅ DELIVERABLES CUỐI CÙNG:                                      │
│  1. Architecture docs (mid-term đã có, cập nhật)                 │
│  2. Final report                                                 │
│  3. Code repo (reproducible)                                     │
│  4. Experiment logs, CSVs, dashboard screenshots                 │
│  5. Demo video                                                   │
│  6. Slides                                                       │
└──────────────────────────────────────────────────────────────────┘
```